

Master's Thesis

Network Security Group, Department of Computer Science, ETH Zurich

**Intent-Conditioned Reinforcement
Learning for Path Selection and
Multipath Optimization in SCION**

by Octave Charrin

Spring 2026

ETH student ID:	24-937-385
E-mail address:	ocharrin@ethz.ch
Supervisors:	Prof. Dr. Adrian Perrig Liwen Xu
Date of submission:	August 25, 2026

Abstract

The Internet forwards packets without regard for why they were sent, so the network optimizes a target that rarely matches what an application actually values. Path-aware architectures such as SCION change this by letting endpoints choose among multiple end-to-end paths, placing the path decision in the hands of the one party that knows the application’s intent. This thesis studies how to make that choice well: how to select the best path – or the best combination of paths – given both the application’s intent and the current state of the network.

We take a single idea from recent congestion-control work, that application intent should be a first-class input which reshapes a network mechanism, and push it into two settings. First, we turn a fixed-objective DQN path selector for SCION into an intent-conditioned one, combining a permutation-equivariant scoring architecture that handles a variable, changing set of candidate paths with feature-wise modulation of path features by an intent vector; a single policy then serves a spectrum of objectives – throughput, latency, trust, probe-frugality – without retraining. We show, analytically and empirically, that conditioning must *modulate* the decision rather than merely accompany it, since an additive signal cancels under the comparison that selects a path. Second, we ask what happens when several paths are used at once and the application is optimized alongside the network: a two-timescale hierarchical pair of reinforcement-learning agents co-optimizes a real-time video call’s encoding bitrate and its per-path traffic split over Multipath QUIC. Here the central finding is that the value of learned multipath scheduling depends sharply on network volatility – nearly negligible when paths are stable, but decisive under path churn and congestion bursts, where controlling the application alone collapses. Together the two parts argue that on a path-aware Internet, application intent should steer every controllable decision between the application and the wire, and they chart what remains to make such control deployable.

Acknowledgements

[TODO: Personal acknowledgements to be written before submission.] I would like to thank my supervisor Liwen Xu for his guidance throughout this thesis, and Prof. Dr. Adrian Perrig for the opportunity to carry out this work in the Network Security Group at ETH Zurich. I am grateful to the members of the group for their feedback, and to the authors of the SCION DQN path-selection and Mortise systems, whose work this thesis builds upon. **[TODO: add thanks to family, friends, and anyone else who supported the work.]**

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Contributions	2
1.2 Organization	3
2 Background	5
2.1 SCION	5
2.2 Conditioning Policies on Application Intent	6
2.2.1 Goal-conditioned reinforcement learning	6
2.2.2 Feature-wise linear modulation	6
2.3 Quality of Experience and Multipath Transport	6
2.3.1 Quality of experience for real-time video	6
2.3.2 Adaptive bitrate and learned control	7
2.3.3 Multipath QUIC	7
2.4 Reinforcement Learning	7
2.4.1 Markov decision processes	7
2.4.2 Deep Q-networks and refinements	7
2.4.3 Soft actor-critic	8
2.4.4 Variable and structured action spaces	8
3 Problem Statement	9
3.1 Problem Formulation	9
3.2 Desired Properties	9
3.3 Assumptions	10
I Intent-Aware Path Selection in SCION	13
4 Design: Intent-Aware Path Selection	15
4.1 Starting Point and Its Limitations	15
4.2 Improved SCION Simulation Environment	16
4.3 Decision Problem: State, Action, Reward	16
4.4 Handling a Variable Path Set: The Scoring Architecture	17

4.5	Intent Conditioning via Feature-Wise Modulation	18
4.5.1	Why concatenation fails	18
4.5.2	Weight-FiLM dueling head	18
4.5.3	Training across intents	18
4.5.4	Relation to Mortise	19
5	Implementation: SCION Path-Selection Pipeline	21
5.1	Agent Family	21
5.2	Evaluation Pipeline	22
5.3	Baselines and Probing Accounting	22
6	Evaluation: Intent-Aware Path Selection	25
6.1	Experimental Setup	25
6.2	Comparison Against Baselines	25
6.3	Intent Alignment	26
6.4	Probing Overhead	26
6.5	Ablation Study	27
6.6	Summary	27
II	Joint Application–Network Optimization over Multipath	29
7	Design: Joint Application–Network Optimization over Multipath	31
7.1	Motivation and Strawmen	31
7.2	Two-Timescale Hierarchical Agents	32
7.3	Observations, Actions, and Rewards	32
7.4	Coupling the Two Agents	33
7.5	Continuous Scoring over a Variable Path Set	33
8	Implementation: Multipath Real-Time Video Testbed	35
8.1	Data-Plane Abstraction and Two Backends	35
8.2	Real-Time Video and Multipath Model	36
8.3	Agents and QoE Surrogate	36
8.4	Known Gaps	37
9	Evaluation: Joint Multipath Optimization	39
9.1	Setup and Baselines	39
9.2	Results: When Does Splitting Help?	39
9.3	Ablation Study	40
9.4	Summary	41

III Synthesis and Outlook	43
10 Analysis	45
10.1 Overhead	45
10.2 Scalability	45
10.3 Generalization	46
10.4 Deployability	46
10.4.1 Deployment Gaps	46
10.4.2 Deployment and Trust Assumptions	47
11 Discussion	49
11.1 Interpretation	49
11.2 Implications	50
11.3 Limitations	50
12 Related Work	51
12.1 Learned Routing and Traffic Engineering	51
12.2 Learned Congestion Control and Multipath Scheduling	51
12.3 Adaptive Bitrate and QoE-Driven Control	52
12.3.1 Mortise: intent-driven tuning of a transport mechanism	52
12.4 Goal-Conditioned and Modulated Policies	52
12.5 Intent-Based Networking	52
13 Conclusion	55
Bibliography	57
A Reproducibility Details	61
A.1 Part I: Reward Profiles	61
A.2 Part I: Agent Hyperparameters	61
A.3 Part II: QoE Weights and SAC Hyperparameters	61
A.4 Supplementary Results	62

1 Introduction

The Internet was designed to deliver packets, not experiences. A router forwards a datagram toward its destination without any notion of *why* the datagram was sent: whether it carries a frame of an interactive video call that will be worthless if it arrives late, a byte of a background software update that only cares about aggregate throughput, or a control message whose loss would be catastrophic. For decades this deliberate ignorance – the end-to-end principle – was a feature. Yet as applications have become more diverse and more demanding, the mismatch between what the network optimizes and what the application actually values has grown into one of the central inefficiencies of modern networking.

Path-aware Internet architectures such as SCION [14] change the terms of this problem. In SCION, endpoints discover *multiple* end-to-end paths to a destination and choose among them, rather than deferring blindly to whatever route BGP happens to install. This turns path selection from a network-internal decision into an application-facing one: for the first time the entity that knows the application’s intent – the endpoint – also holds the lever that determines how traffic is carried. The question this thesis studies is the natural consequence of that shift:

How should an endpoint choose the best path – or the best combination of paths – given both the intention of the application and the current state of the network?

Answering this question is hard for three compounding reasons. First, “best” is not a fixed target. A path that is best for a bulk transfer (high goodput, tolerant of latency) is often a poor choice for a video call (which prizes low latency and jitter and will trade raw throughput to get them). Any single hand-tuned objective is therefore wrong for some application, and training a separate policy for every conceivable objective does not scale. Second, *the decision space is dynamic and unbounded*. The set of available paths between two hosts differs from pair to pair and changes over time as links appear, fail, congest, or recover; a policy that assumes a fixed, enumerated action set cannot cope. Third, *the state is only partially and expensively observable*. Fresh per-path measurements cost probing overhead, so a good policy must decide well while measuring little.

This thesis attacks the question in two movements, unified by a single idea borrowed from recent work on congestion control. The Mortise system [17] shows that a transport mechanism can be steered to maximize an application’s quality of experience (QoE) by treating the application’s *intent* as an explicit input that reshapes how the mechanism is tuned, rather than as a fixed constant baked into the algorithm. We take the same stance and push it into two new settings.

Part I: steering *which* path. We build on the DQN-based SCION path selector of Bourak et al. [3] and rework it into an intent-conditioned path-selection agent. We improve the underlying SCION simulation environment, replace the fixed-action DQN with a permutation-equivariant *path-scoring* architecture that handles a variable number of candidate paths, and – the key contribution – condition the agent on a compact representation of application intent using *feature-wise linear modulation* (FiLM) [13]. A single trained policy can then serve a throughput-hungry transfer, a latency-sensitive call, or any point in between, simply by changing the intent vector at inference time, without retraining. This is the path-selection analogue of Mortise’s intent-driven parameter tuning.

Part II: steering *how much* of each path, and *what* the application sends. Part I chooses a single path. But SCION’s multipath nature invites a more ambitious question: what if we use several paths at once, and co-optimize the application itself? Where Mortise tunes the congestion-control algorithm to the application, we tune both the *application* (the video encoder’s target bitrate) and the *network* (the split of traffic across paths) jointly. Using a Multipath-QUIC [5] transport carrying a real-time WebRTC-style video flow, we design a hierarchical pair of reinforcement-learning agents – a slow application agent that adapts bitrate and a fast transport agent that schedules bytes across paths – coupled so that each accounts for the other. This closes the loop from intent all the way to what the application chooses to produce. Because Part II’s Multipath QUIC transport rides on top of the same path-aware substrate, it is best read as *multipath transport enabled by SCION’s native path multiplicity*: SCION supplies the several disjoint paths, and the agents decide how to use them.

Throughout, a set of questions guides the reader. *Can one policy serve many intents without retraining, and does conditioning actually change its decisions or merely its rhetoric? When is it worth splitting traffic across paths at all, and when does a single good path suffice? Does co-optimizing the application with the network beat optimizing either alone? And how far is any of this from something one could deploy?* We will see that the answers are often not the obvious ones – for instance, that multipath scheduling can be nearly worthless in a stable network yet decisive in a volatile one.

1.1 Contributions

This thesis makes the following contributions.

1. **An intent-conditioned path-selection agent for SCION.** We extend the DQN path selector of Bourak et al. [3] with (i) an improved, higher-fidelity SCION simulation environment driven by realistic beaconing and multi-day traffic, (ii) a permutation-equivariant path-scoring network that supports a variable and changing set of candidate paths, and (iii) a FiLM-based conditioning

mechanism that lets one policy realize a spectrum of application intents without retraining (Chapters 4 and 5).

2. **Evidence that intent conditioning must modulate, not merely append.** We show analytically and empirically why naively concatenating an intent vector to the state fails to change a greedy selector’s decisions, and why feature-wise modulation is the appropriate mechanism (Chapter 4).
3. **A hierarchical agent pair for joint application–network optimization over multipath.** We design and implement a two-timescale reinforcement-learning system that co-optimizes video bitrate (application) and per-path traffic split (network) over Multipath QUIC, realized on both a packet-level NS-3 backend and a fast analytical backend behind a common interface (Chapters 7 and 8).
4. **An evaluation and analysis of *when each mechanism pays off*.** We characterize the conditions under which intent conditioning and multipath scheduling deliver value – and the conditions under which simpler baselines suffice – and we assess the overhead, scalability, generalization, and deployability of both systems (Chapters 10 and 11).

1.2 Organization

Chapter 2 reviews the background this thesis builds on: the SCION architecture, the reinforcement-learning machinery (DQN and its refinements, soft actor-critic, variable action spaces, and FiLM-based conditioning), and the QoE and multipath-transport concepts used in Part II. Chapter 3 states the problem precisely, lists the properties a good solution must have, and makes explicit the assumptions and trust model.

The body is then organized in three parts. **Part I** covers intent-aware single-path selection in SCION: its design (Chapter 4), implementation (Chapter 5), and evaluation (Chapter 6). **Part II** covers joint application–network optimization over multipath: design (Chapter 7), implementation (Chapter 8), and evaluation (Chapter 9). **Part III** draws the two together: Chapter 10 analyzes overhead, scalability, generalization, and deployability; Chapter 11 discusses what the results mean; Chapter 12 places the work in the research landscape; and Chapter 13 concludes.

2 Background

This chapter reviews the systems and techniques the thesis builds on. Section 2.1 introduces the SCION path-aware Internet architecture and the path-selection opportunity it creates. Section 2.4 recalls the reinforcement-learning (RL) machinery used in both parts: the Markov decision process, deep Q -networks and their standard refinements, soft actor-critic, and the handling of variable action spaces. Section 2.2 covers the mechanisms for conditioning a policy on application intent – goal-conditioned RL and feature-wise linear modulation – and situates them against the Mortise congestion-control tuner that inspires this work. Section 2.3 introduces the quality-of-experience and multipath-transport concepts specific to Part II.

2.1 SCION

SCION [14] is a next-generation, path-aware Internet architecture designed for high availability and transparency of the forwarding path. Its defining departure from today’s Internet is that *end hosts, not intermediate routers, choose the path a packet takes*. The network is organized into *isolation domains* (ISDs), each governed by a set of core autonomous systems (ASes). Path segments are discovered by a beaconing process: core ASes flood *path-segment construction beacons* (PCBs) that accumulate hop information as they propagate, giving rise to three segment types – *up* segments from a leaf AS to its ISD core, *core* segments between core ASes across ISDs, and *down* segments from a core to a destination leaf. An end host combines an up, an (optional) core, and a down segment into a complete forwarding path and embeds it in the packet header.

Two properties of SCION matter for this thesis. First, **path multiplicity**: a source–destination pair is typically connected by *several* distinct paths that differ in latency, available bandwidth, loss, and the ASes they traverse. This is precisely the raw material a path-selection agent needs, and the same multiplicity is what makes multipath transport natural in Part II. Second, **path transparency and trust**: because the full AS-level path is visible to the endpoint, an agent can reason about properties beyond raw performance – for instance a *trust* score that reflects how much confidence to place in a path’s advertised or measured quality. We exploit this in Part I, where the reward balances measured goodput against a trust term. The flip side of endpoint path choice is a coordination problem: whereas a router runs one routing process, every SCION host must decide for itself, ideally without

flooding the network with probes. Minimizing that probing overhead is an explicit objective throughout Part I.

2.2 Conditioning Policies on Application Intent

2.2.1 Goal-conditioned reinforcement learning

A policy that must serve many objectives can be made a function of the objective itself. In goal-conditioned RL, the policy $\pi(a \mid s, g)$ and value function are conditioned on a goal g , so that one network represents a whole family of behaviors. Universal value function approximators (UVFAs) [15] formalize this by learning $V(s, g)$ jointly over states and goals, and hindsight experience replay [2] makes it sample-efficient by relabeling failed trajectories with the goals they did achieve. In this thesis the “goal” is application intent, encoded as a vector of reward weights: the agent is told, at both training and inference time, how much it should care about throughput versus latency versus trust, and is expected to act accordingly.

2.2.2 Feature-wise linear modulation

How the intent vector influences the network turns out to matter a great deal. The mechanism we adopt is feature-wise linear modulation (FiLM) [13]: a small conditioning network maps the intent vector to per-feature scale and shift parameters (γ, β) , which affinely transform an intermediate feature representation h as $\text{FiLM}(h) = (1 + \gamma) \odot h + \beta$. Because the modulation is multiplicative, it can change the *relative* importance of different path features – amplifying latency for a real-time intent, amplifying bandwidth for a bulk intent – and therefore change which path scores highest. As we argue in Chapter 4, this is the property that a mere concatenation of the intent vector onto the state lacks: appending a constant to every path’s input tends to shift all path scores together, leaving the $\arg \max$ unchanged.

2.3 Quality of Experience and Multipath Transport

2.3.1 Quality of experience for real-time video

Part II targets interactive video, whose value to a user is captured by quality-of-experience (QoE) rather than any single network metric. Perceptual quality at a given encoding is commonly summarized by VMAF [10], a learned metric that predicts subjective video quality; interactivity adds sensitivity to end-to-end latency, jitter, and frame loss, since a late frame in a live call is a lost frame. Part II’s

reward composes a VMAF-based quality term with penalties for latency, jitter, and loss, making the QoE objective explicit and tunable.

2.3.2 Adaptive bitrate and learned control

The application-side lever in Part II is adaptive bitrate (ABR) control: choosing the video encoder’s target rate to match available capacity. Learned ABR controllers such as Pensieve [11] showed that RL can outperform hand-crafted heuristics for buffered video streaming. Our setting differs in two ways: it is real-time (no large playback buffer to hide variance), and the bitrate decision is coupled to a simultaneous *network-side* decision about how to use multiple paths.

2.3.3 Multipath QUIC

QUIC [8] is a modern, encrypted, multiplexed transport running over UDP. Multipath QUIC [5] extends it to send a single connection’s data across several network paths at once, in the spirit of Multipath TCP [6] but with QUIC’s flexibility and user-space deployability. Over a path-aware substrate like SCION, the several paths are readily available from path selection, which makes MPQUIC a natural transport for exploiting SCION’s path multiplicity. The open question MPQUIC raises – how to *schedule* bytes across heterogeneous paths – is one of the two decisions Part II learns.

2.4 Reinforcement Learning

2.4.1 Markov decision processes

We model path and rate control as sequential decision-making in a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where at each step the agent observes a state $s \in \mathcal{S}$, takes an action $a \in \mathcal{A}$, receives a scalar reward $r(s, a)$, and transitions to s' with probability $P(s' | s, a)$. The agent seeks a policy π that maximizes the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ with discount $\gamma \in [0, 1)$. The reward function is where application intent enters: by changing what the agent is rewarded for, we change what “best” means.

2.4.2 Deep Q-networks and refinements

The action-value function $Q^\pi(s, a) = \mathbb{E}[\sum_t \gamma^t r_t | s_0 = s, a_0 = a]$ gives the expected return of taking a in s and following π thereafter. A deep Q-network (DQN) [12] approximates the optimal Q^* with a neural network trained to minimize the temporal-difference error against a slowly updated *target* network, sampling transitions from a replay buffer to decorrelate them. Part I uses three standard refinements. *Double DQN* [19] decouples action selection from action evaluation

in the bootstrap target, reducing the overestimation bias of vanilla DQN. The *dueling* architecture [20] factors $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$ into a state-value stream V and an advantage stream A , which helps when many actions have similar value – as when several paths are comparably good. *Prioritized experience replay* (PER) [16] samples high-error transitions more often, correcting the resulting bias with importance-sampling weights.

2.4.3 Soft actor-critic

Part II controls *continuous* quantities – a target bitrate and a split ratio over paths – for which value-based DQN is ill-suited. We use soft actor-critic (SAC) [7], an off-policy, maximum-entropy actor-critic method. SAC augments the reward with a policy-entropy term, $\mathbb{E}[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)))]$, which encourages exploration and robustness; it trains twin Q -critics to curb overestimation and can tune the temperature α automatically against a target entropy. SAC’s sample efficiency and stability make it a good fit for the expensive, noisy rollouts of a packet-level network simulator.

2.4.4 Variable and structured action spaces

A subtle but central difficulty is that the number of candidate paths is not fixed: it varies across source–destination pairs and over time. A classical DQN with a fixed output head cannot represent this. The remedy, used in both parts, is a *scoring* architecture that is *permutation-equivariant* over the path set: a shared network maps each path’s feature vector (together with a global context) to a scalar score, and the scores are compared – by arg max for discrete selection in Part I, or by a softmax to produce a traffic split in Part II. Invalid or absent paths are masked out. Aggregating a set of per-element embeddings in a way that is invariant to their ordering is the Deep Sets construction [23], which we use in Part II’s critic. This design decouples the policy from the cardinality of the action set and lets a single network generalize across topologies with different numbers of paths.

3 Problem Statement

This chapter states the problem the thesis addresses, the properties a good solution must exhibit, the assumptions under which we operate, and the trust model that replaces the classical attacker model for this performance-oriented work.

3.1 Problem Formulation

Consider an end host that must carry an application's traffic toward a destination over a path-aware network. At any moment the host has access to a set of candidate paths $\mathcal{P} = \{p_1, \dots, p_N\}$, each described by measurable (and partly stale) properties – latency, available bandwidth, loss, hop count, a trust score – and the application expresses an *intent* g : a statement of what it values, such as “maximize goodput” or “minimize latency and jitter.” The network's state evolves over time as cross-traffic, congestion, and topology change. The host must repeatedly decide how to serve the application:

- In **Part I**, the decision is a *single path* $p^* \in \mathcal{P}$ to carry the flow, chosen to maximize the application's intent-weighted objective while spending as little probing effort as possible.
- In **Part II**, the decision is richer: a *split* $(w_1, \dots, w_N)$ of the flow's traffic across the paths *and* the application's own *sending rate* (video bitrate), chosen jointly to maximize real-time video QoE.

In both cases the underlying question is the one posed in Chapter 1: *choose the best use of the available paths given the application's intent and the network's state*. The technical crux is that neither the objective nor the action set is fixed – the objective is dictated by an intent that can change from flow to flow, and the action set is a path collection that changes across pairs and over time.

3.2 Desired Properties

A satisfactory solution should have the following properties, which also define the quality metrics used in the evaluation (Chapters 6, 9, and 10).

Intent alignment. The chosen action should maximize the objective the application actually cares about. Concretely, a policy conditioned on intent g should

make decisions that score well under g – and *measurably differently* for different g . A policy that ignores its intent input has failed this property even if its average reward looks reasonable.

No per-objective retraining. A single trained policy should serve the whole spectrum of intents, including intents not seen at training time, rather than requiring a separately trained model for each objective. This is what makes the approach deployable at Internet scale.

Dynamic-action handling. The policy must accept a variable and changing number of candidate paths without retraining or architectural change, and remain invariant to the arbitrary ordering of those paths.

Low overhead. Decisions should be cheap. In Part I this means minimizing active probing of paths (a first-class term in the reward); in both parts it means bounded per-decision inference cost compatible with the control loop's timescale.

Generalization. Performance should hold across topologies, traffic conditions, and path counts not seen during training, degrading gracefully rather than catastrophically off-distribution.

Robustness to volatility. Under non-stationary conditions – congestion bursts, path churn, regime shifts – the policy should retain its advantage over static heuristics, which is precisely where learned control is expected to pay off.

3.3 Assumptions

We make the following assumptions explicit.

1. **Path-aware substrate.** The host can enumerate multiple end-to-end paths to a destination and direct traffic onto a chosen path or split, as SCION provides. We do not assume control over intra-AS routing.
2. **Observable per-path metrics.** Per-path latency, loss, and a bandwidth estimate are measurable, at a probing cost that the agent must economize (Part I) or are available from transport feedback such as acknowledgements and RTT samples (Part II).
3. **Intent as a QoS preference.** Application intent is expressed as a compact vector – a set of reward weights over throughput, latency, jitter, loss, and trust (Part I), or the fixed QoE weighting of a real-time video call (Part II). We do not attempt to *infer* latent intent from traffic; we assume the application (or a shim on its behalf) can declare it.

4. **Stationarity within a decision epoch.** Network conditions are assumed roughly stable over the short interval of a single decision, though they may drift or shift abruptly between epochs; the non-stationary regimes we study (Chapter 9) stress exactly this assumption.

Part I

Intent-Aware Path Selection in SCION

4 Design: Intent-Aware Path Selection

Part I asks how a single path should be chosen from a SCION host’s candidate set so as to serve the running application’s intent under a changing network. This chapter presents the design. We start from the DQN path selector of Bourak et al. [3] and identify three limitations we set out to remove (§4.1). We then describe the improved simulation environment (§4.2), the state, action, and reward that define the decision problem (§4.3), the permutation-equivariant path-scoring architecture that handles a variable path set (§4.4), and finally the FiLM-based intent conditioning that is the chapter’s main contribution (§4.5).

4.1 Starting Point and Its Limitations

Bourak et al. [3] frame SCION path selection as a reinforcement-learning problem and train a DQN to pick a path from per-path measurements of latency, loss, and bandwidth, rewarding a weighted combination of goodput and trust while charging for probing. They report oracle-level selection quality at a fraction of the probing cost of measuring every path. This validates the RL framing and gives us a reward structure to build on, but three design choices limit it.

Fixed action space. A vanilla DQN has one output per action, so the number of paths must be fixed and known. Real source–destination pairs have different, time-varying path counts; padding to a maximum wastes capacity and still caps the topology size.

Single baked-in objective. The reward weights are constants chosen before training. The resulting policy embodies *one* trade-off between goodput and trust; a different application intent demands a different, separately trained model.

Environment fidelity. Capturing when path selection actually matters requires paths that genuinely share bottlenecks and conditions that vary over realistic timescales, so that a good choice at one hour is a poor choice at another.

Our design removes all three: a scoring architecture for the first, FiLM conditioning for the second, and an improved environment for the third. A strawman for the second limitation – simply appending the reward weights to the state of the existing DQN – is instructive precisely because it *fails*, as we show in §4.5; that failure motivates the modulation-based design.

4.2 Improved SCION Simulation Environment

Training and evaluating a path selector requires an environment in which the “right” path changes with conditions. We construct one in several stages.

Topology. AS-level topologies are generated with BRITE and converted into SCION structures – ISDs, core and non-core ASes, and inter/intra-ISD links – so that the graph has realistic degree and locality structure rather than being hand-drawn.

Control plane. A beaconing simulation propagates path-segment construction beacons top-down within each ISD and across the core mesh between ISDs, yielding the up/core/down segments that compose into end-to-end paths. Peer links are excluded. The result is, for each source–destination pair, a realistic set of candidate paths of varying length and quality – the agent’s action set.

Traffic and link dynamics. We drive the network with 28 days of hourly demand: foreground flows for every routable pair plus randomized background traffic, modulated by diurnal and weekly factors. Crucially, load is aggregated *per link*, so paths that share a bottleneck link contend for its capacity. This is what makes selection non-trivial: the utilization and effective bandwidth of a path depend on what every other flow is doing at that hour, and the best path at 3 a.m. is often congested at 8 p.m.

Probing model. Fresh metrics are not free. The environment charges a latency probe a small per-hop cost and a full bandwidth probe a substantially larger one, and it credits the agent with having probed only the path(s) it actually inspects. This turns “measure less” into a real objective rather than an afterthought, and lets us compare the learned selector’s probing footprint against baselines that probe exhaustively.

4.3 Decision Problem: State, Action, Reward

State. The agent observes a *global* context and a *per-path* feature matrix. The global context summarizes time and aggregate conditions – normalized day-of-week and hour-of-day, mean path utilization, mean trust, and the fraction of paths that are congested – plus a compact encoding of the source and destination. Each candidate path contributes a feature row: normalized latency, loss rate, hop count, a bandwidth ratio, utilization, a static-bandwidth term, and a trust score. Because the number of rows N varies, the state is naturally a (global, path-matrix) pair rather than a flat vector.

Action. The action is the choice of one path, i.e. an index into the current candidate set. Because N varies, the action is realized as an $\arg \max$ over per-path scores (§4.4) with invalid entries masked, rather than as a fixed categorical head.

Reward. We adopt and generalize the goodput–trust reward of Bourak et al. [3]. For the chosen path, let $\text{goodput} \in [0, 1]$ be the achieved bandwidth normalized by an auto-detected per-topology capacity, and define a trust term from loss and delay,

$$\text{trust} = \max(0, 1 - (w_3 \text{loss} + w_4 \text{delay})), \quad (4.1)$$

with delay the latency capped at 100 ms and normalized. The base reward trades goodput against trust and is shifted to $[-1, 1]$, and a probing penalty is subtracted:

$$r = \underbrace{2(w_1 \text{goodput} + w_2 \text{trust}) - 1}_{\text{base reward}} - w_{\text{probe}} \cdot \text{probe_penalty}, \quad (4.2)$$

clamped to $[-1, 1]$, where probe_penalty is the probing cost normalized to $[0, 1]$. The five weights $\mathbf{w} = (w_1, w_2, w_3, w_4, w_{\text{probe}})$ are exactly the *application intent*: default balanced values are $(0.7, 0.3, 0.5, 0.5, 0.05)$, but a throughput-hungry intent pushes w_1 toward 1, a loss-averse intent raises w_2 and w_3 , a latency-sensitive intent raises w_4 , and a probe-frugal intent raises w_{probe} . The design goal of §4.5 is a *single* policy that behaves correctly for *any* setting of $\mathbf{w}$.

4.4 Handling a Variable Path Set: The Scoring Architecture

To make the policy independent of the path count, we score each path with a *shared* network and select by comparison, following the permutation-equivariant design of §2.4.4. The global context is broadcast and concatenated to every path’s feature row, and a shared multilayer perceptron maps each (global, path_i) pair to a scalar Q -value. Selection is $\arg \max_i Q_i$; absent or invalid paths are masked to -10^9 so they can never be chosen and do not distort training targets.

We use a *dueling* scoring head (§2.4): a value stream reads the global context alone to produce $V(s)$, an advantage stream reads the per-path features to produce $A(s, i)$, and the two combine as $Q_i = V(s) + (A(s, i) - \text{mean}_{j \in \text{valid}} A(s, j))$ with the mean taken only over valid paths. Because many paths are often comparably good, separating “how good is this situation” from “which path is marginally better” stabilizes learning. Double DQN targets and prioritized replay (§2.4) further stabilize and accelerate training. The scoring design also delivers the dynamic-action property directly: the same weights serve a pair with three paths and a pair with thirty, and are invariant to how the paths happen to be ordered.

4.5 Intent Conditioning via Feature-Wise Modulation

The central contribution of Part I is making one scoring policy serve every intent $\mathbf{w}$. We condition the network on $\mathbf{w}$, but the *mechanism* of conditioning is what makes or breaks it.

4.5.1 Why concatenation fails

The obvious approach is to append $\mathbf{w}$ to the state. But consider a scoring policy: the intent is part of the *global* context, which is broadcast *identically* to every path before scoring. Adding the same constant vector to every path’s input tends to move all path scores by a similar amount – a common-mode shift – which leaves the $\arg \max$, and hence the chosen path, essentially unchanged. Formally, if conditioning enters only additively and symmetrically across paths, it cancels under the comparison that defines the action. The policy can thus appear to “take intent as input” while its behavior is invariant to it. This is not a hypothetical; it is the failure mode our ablation in Chapter 6 is designed to expose, and it is why a naive goal-conditioned baseline is inadequate.

4.5.2 Weight-FiLM dueling head

The fix is to let the intent modulate the *per-path* features multiplicatively, so that it changes the relative ranking of paths rather than a shared baseline. A small conditioning network maps the intent vector to feature-wise scale and shift parameters,

$$(\gamma, \beta) = \text{FiLM}_{\theta}(\mathbf{w}), \quad \tilde{f}_i = (1 + \gamma) \odot f_i + \beta, \quad (4.3)$$

where f_i is path i ’s feature row and $\tilde{f}_i$ the modulated features fed to the advantage stream. A latency-sensitive intent can learn a γ that amplifies the latency feature, so paths differ more sharply in their latency contribution and a low-latency path wins; a throughput-hungry intent can instead amplify bandwidth. Because the modulation is per-feature and multiplicative, it acts *differently* on paths with different feature values and therefore genuinely reorders them – exactly what concatenation cannot do. The value stream continues to read only the topology/time context, keeping the state-value estimate intent-agnostic while the advantage – the part that decides *which* path – is intent-modulated.

4.5.3 Training across intents

To learn a policy that spans the intent space we train across a set of *reward profiles* – named weight vectors spanning balanced, throughput-maximizing, trust/loss-averse, latency-sensitive, and probe-frugal extremes. Each training episode samples a profile, sets the environment’s reward weights to it, and feeds the same weights to the FiLM head, so the agent simultaneously experiences

the reward for an intent and the encoding of that intent. Profiles are scheduled in a stratified fashion so every intent receives balanced coverage. At inference, presenting a new $\mathbf{w}$ – including interpolations between trained profiles – yields the corresponding behavior without any retraining.

4.5.4 Relation to Mortise

This is the path-selection instance of Mortise’s principle [17]. Where Mortise maps an application’s revealed QoS preference to congestion-control *parameters*, our FiLM head maps an explicit intent vector to a *modulation* of path features that determines which path is chosen. In both, intent is a first-class input that reshapes a network mechanism, not a constant compiled into it. Chapter 7 pushes the same principle one step further, letting intent shape not only the network’s choice but the application’s own output.

5 Implementation: SCION Path-Selection Pipeline

This chapter describes how the Part I design is realized: the agent family and their architectures (§5.1), the end-to-end evaluation pipeline that turns a topology into trained models and comparable results (§5.2), and the baselines and probing accounting against which the learned selectors are measured (§5.3). The implementation is in Python (PyTorch for the agents) and is organized so that each pipeline stage passes data to the next through timestamped run directories.

5.1 Agent Family

The codebase implements a family of agents of increasing capability, which also serve as the ablation ladder in Chapter 6.

Flat DQN (simple / enhanced). A conventional fixed-action DQN over the 5-dimensional global state. The *simple* variant is a small multilayer perceptron with ϵ -greedy exploration and a periodic target network; the *enhanced* variant adds Double DQN [19], a dueling head [20], prioritized experience replay [16], soft target updates, and action masking. These agents cannot handle a variable path set and stand in for the original design [3].

Path-scoring DQN (simple / enhanced). The permutation-equivariant design of §4.4: a shared per-path scorer producing one Q -value per path, with variable N handled by padding to the batch maximum and masking. The enhanced variant uses the dueling scoring head, Double DQN, and prioritized replay. This family delivers the dynamic-action property.

Conditional path-scoring DQN. The recommended agent, adding the weight-FiLM dueling head of §4.5. Its global state is extended with the five-dimensional encoded intent vector, and the FiLM sub-network modulates the per-path features feeding the advantage stream. Two intent encodings are supported and recorded in the checkpoint – the raw weights, or a policy-friendly rescaling to $[-1, 1]$ for smoother gradients.

Training details. Prioritized replay uses priority exponent $\alpha = 0.6$ with the importance-sampling exponent β annealed from 0.4 to 1.0. Target networks use

soft (Polyak) updates. Invalid actions are masked to a large negative value both when acting and when forming bootstrap targets, so masked paths never contribute to the loss. The conditional agent is trained across reward profiles on a stratified schedule (§4.5), so that within a single training run the network sees every intent an equal number of times.

5.2 Evaluation Pipeline

The system is driven by a six-stage pipeline, each stage writing artifacts consumed by the next:

1. **Topology generation** – BRITE produces an AS graph, converted into SCION roles, ISDs, and links.
2. **Beaconing** – PCB propagation discovers up/core/down segments and composes candidate paths per source–destination pair.
3. **Traffic simulation** – 28 days of hourly demand with per-link aggregation, producing the time-varying link states the environment replays.
4. **Training** – all agent variants are trained on the resulting environment; checkpoints and per-episode statistics are saved.
5. **Evaluation** – every method (learned and baseline) is run over held-out pairs, hours, and intents, logging reward, achieved latency and bandwidth, probing overhead, and selection time.
6. **Figures** – results are rendered in a consistent style for the report.

Running the whole pipeline is a single command; intermediate artifacts (topology, path store, link states, checkpoints, results tables) live under a timestamped run directory so that any stage can be re-run or inspected in isolation. Key knobs – topology size, number of training episodes, the cap on training pairs, and learning hyperparameters – are exposed as configuration so that experiments can be scaled up or down.

5.3 Baselines and Probing Accounting

Baselines. Six heuristic selectors provide the comparison points: shortest-path, widest-path, lowest-latency, ECMP, the SCION default selection, and random. They span the spectrum from naive (random) to strong single-metric heuristics (widest for throughput, lowest-latency for delay), so that the learned selectors must beat not just a weak baseline but the *right* heuristic for each intent.

Probing accounting. A fair comparison must charge each method for the measurements it needs. The environment therefore records, per decision, which paths a method probed and at what cost – latency probes being cheap and full bandwidth probes expensive (§4.2). Heuristics that require a metric on every path (e.g. widest-path) incur the cost of probing all paths, whereas the learned selector is charged only for the paths it actually inspects. This makes the probing-overhead reduction a directly measurable quantity rather than an assumed benefit, and is the mechanism by which we reproduce and test the near-oracle-quality-at-low-probing claim of Bourak et al. [3].

6 Evaluation: Intent-Aware Path Selection

This chapter evaluates the Part I selectors. It states the experimental setup and metrics (§6.1), then reports results against the six baselines (§6.2), the intent-alignment behavior of the conditional agent (§6.3), the probing-overhead reduction (§6.4), and an ablation ladder isolating each design element (§6.5). Several results tables depend on experiment runs that are still in progress; those are marked and left as structured placeholders so that the remaining experimental work is well defined.

6.1 Experimental Setup

Environments. Experiments use BRITE-generated SCION topologies of varying size, each driven by the 28-day hourly traffic model with per-link aggregation (§4.2). Training and evaluation are separated by holding out a disjoint set of source–destination pairs and hours, so that reported numbers reflect generalization rather than memorization. *[TODO: state the exact topology sizes, number of ASes/links, number of pairs, and train/test split actually used, once the full pipeline run completes.]*

Intents. The conditional agent is trained across the reward profiles of §4.5 (balanced, throughput, trust/loss-averse, latency, probe-frugal) and evaluated on those profiles plus held-out interpolated intents to test generalization across the intent space.

Metrics. For each method we report: the intent-weighted reward of Eq. (4.2); the achieved per-flow latency and bandwidth; the probing overhead (in normalized probe cost); and the per-decision selection time. Intent alignment is assessed both by reward under each intent and by *behavioral divergence* – how much the chosen paths change as the intent changes.

6.2 Comparison Against Baselines

We compare the conditional path-scoring agent against the six heuristics of §5.3, per intent. The expectation, following [3], is that the learned selector matches or

beats the *best heuristic for each intent* while using far less probing, and – unlike any single heuristic – does so for *all* intents with one model.

Placeholder — Table: mean intent-weighted reward (and achieved latency/bandwidth) for the conditional agent vs. shortest-path, widest-path, lowest-latency, ECMP, SCION-default, and random, one column block per reward profile.

[TODO: populate the baseline comparison table from the stage-05 evaluation results; highlight, for each intent, which heuristic is strongest and whether the learned agent matches or exceeds it.]

6.3 Intent Alignment

The defining test of the design is whether one conditioned policy actually *changes its decisions* with intent. We evaluate the same trained conditional agent under each intent on a fixed set of pairs/hours and measure (i) reward under the matching intent versus reward under a mismatched intent, and (ii) how the distribution of chosen paths shifts – e.g. toward low-latency paths under a latency intent, toward high-bandwidth paths under a throughput intent.

Placeholder — Figure: for a fixed set of decision situations, the chosen-path metric distributions (latency, bandwidth, trust) under each intent, showing the policy re-ranking paths as intent changes.

[TODO: report the intent×intent reward matrix (diagonal should dominate) and the behavioral-shift statistics that confirm the FiLM head steers decisions.]

6.4 Probing Overhead

Using the probing accounting of §5.3, we compare the learned selector’s probing footprint against exhaustive-probing heuristics at matched selection quality, reproducing and testing the near-oracle-quality-at-reduced-probing result of [3].

Placeholder — Figure: selection quality vs. probing cost, learned selector vs. exhaustive baselines and an oracle.

[TODO: report the probing-cost reduction at matched reward, and the quality gap to the oracle.]

6.5 Ablation Study

We isolate the contribution of each design element by walking up the agent ladder of §5.1:

1. **Flat vs. scoring.** Does the permutation-equivariant scoring architecture improve over a fixed-action DQN, and does it generalize across path counts that a flat agent cannot even represent?
2. **Scoring vs. conditional.** Does adding intent conditioning let one model match a suite of per-intent-trained scoring models, avoiding per-objective retraining?
3. **FiLM vs. concatenation.** The critical ablation: replace the FiLM head with plain concatenation of the intent vector and confirm the failure predicted in §4.5.1 – that behavior becomes (near-)invariant to intent even though reward on the training intent may look acceptable.

Placeholder — Table: reward and intent-behavioral-divergence for flat, scoring, conditional-concat, and conditional-FiLM agents, across intents.

[TODO: populate the ablation table; the key expected finding is that concatenation shows near-zero behavioral divergence across intents while FiLM shows large divergence at comparable or better reward.]

6.6 Summary

[TODO: write the 2–3 sentence takeaway once the tables are filled: one conditioned scoring policy serves the full intent spectrum, matches the best per-intent heuristic while probing far less, and requires modulation (not concatenation) to make conditioning effective.]

Part II

Joint Application–Network Optimization over Multipath

7 Design: Joint Application–Network Optimization over Multipath

Part I chose a single path for a flow. Part II starts from a more ambitious premise. SCION offers not one path but several, so why commit to one? And if Mortise [17] can tune a *transport mechanism* to the application, why not tune the *application itself* in the same breath? This chapter designs a system that (i) uses multiple paths at once via Multipath QUIC [5] and (ii) jointly optimizes what the application sends – the video encoder’s target bitrate – and how the network spreads it – the per-path traffic split – to maximize the quality of experience of a real-time video call.

Section 7.1 motivates the joint, multipath framing and the strawmen it improves on. Section 7.2 presents the two-timescale hierarchical agent pair. Section 7.3 specifies the observations, actions, and rewards. Section 7.4 explains the coupling that keeps the two agents cooperating rather than working at cross purposes. Section 7.5 extends the scoring idea of Part I from a discrete choice to a continuous split over a variable, possibly changing, path set.

7.1 Motivation and Strawmen

Interactive video is a demanding, ubiquitous workload with no playback buffer to hide variance: a frame that misses its deadline is simply lost. Its QoE depends on two decisions that are usually made in isolation. The *application* chooses an encoding bitrate – too high and the network drops frames, too low and quality suffers. The *network* chooses how to carry the resulting bytes – over one path or spread across several. Optimizing either alone is a strawman:

Application-only. Adapt bitrate to the aggregate network but carry it over a fixed or naive path assignment. This wastes SCION’s path diversity and, as our results show, collapses when the paths it depends on degrade.

Network-only. Schedule cleverly across paths but treat the offered load as exogenous. The scheduler cannot relieve congestion it is itself creating by admitting too high a bitrate.

The two decisions are coupled through the same bottleneck: bitrate sets the load the scheduler must place, and the scheduler’s success sets the effective capacity the bitrate controller sees. Optimizing them jointly is the design’s premise. The challenge is that they naturally operate on *different timescales* – an encoder should

not change bitrate every frame, but a scheduler must react per frame – which motivates a hierarchy rather than a single monolithic agent.

7.2 Two-Timescale Hierarchical Agents

We split control into two cooperating agents that act at different cadences, a “brain and body” decomposition.

Application agent (slow). Every 1 s (once per 30 frames at 30 fps) it observes an aggregate summary of the connection and outputs a single target encoder bitrate in the range 300–6000 kbit/s. It is deliberately *horizon-agnostic* – it never observes how far along the call is – so the same policy applies to a call of any length, an essential property for a controller meant to run in unbounded real sessions.

Transport agent (fast). Every frame (≈ 33 ms) it observes per-path transport state *and the application agent’s current target bitrate*, and outputs a split of that frame’s bytes across the available paths.

Both are trained with soft actor-critic (§2.4), whose continuous action support and sample efficiency suit these controls and the cost of simulator rollouts.

7.3 Observations, Actions, and Rewards

Application observation and action. The application agent sees a small fixed-size vector: normalized current bitrate, smoothed RTT, jitter, loss, and throughput. Its action is one continuous scalar in $[-1, 1]$, mapped affinely to a target bitrate in 300–6000 kbit/s.

Transport observation and action. The transport agent sees a global block (target bitrate, RTT, loss, throughput) and, for each path, a feature vector (congestion window, smoothed RTT, buffer occupancy, per-path throughput, per-path loss). Its action is an N -vector passed through a softmax to yield a split $(w_1, \dots, w_N)$ that is non-negative and sums to one – a valid allocation by construction.

Rewards. The rewards encode the QoE objective and the division of responsibility between the agents. The application’s per-window QoE reward composes a quality term with penalties:

$$\text{QoE} = a \cdot \frac{\text{VMAF}(\cdot)}{100} - b \cdot \widetilde{\text{lat}} - c \cdot \widetilde{\text{jit}} - d \cdot \text{loss}, \quad \text{clipped to } [-2, 1], \quad (7.1)$$

where $\widetilde{\text{lat}}$ and $\widetilde{\text{jit}}$ are latency and jitter normalized (by 200 ms and 50 ms) and clipped, and the default weights are $(a, b, c, d) = (1, 0.5, 0.5, 1)$. The quality term

uses a VMAF surrogate (Chapter 8); when a learned surrogate that already accounts for loss is used, the explicit loss penalty is dropped to avoid double-counting. The transport agent receives a per-frame reward that credits delivery without quality, since it does not control bitrate:

$$R_{\text{transport}} = (1 - \text{loss}) - b \cdot \widetilde{\text{lat}} - c \cdot \widetilde{\text{jit}}, \quad \text{clipped to } [-2, 1]. \quad (7.2)$$

7.4 Coupling the Two Agents

The design’s subtlety is keeping the agents cooperative. Two mechanisms couple them.

Observation coupling. The transport agent observes the application’s current target bitrate. It therefore schedules *knowing* how much load it must place, and can, for instance, spread an aggressive bitrate across more paths rather than overrunning one. This is what makes the pair hierarchical rather than two agents optimizing in ignorance of each other.

Reward decomposition. The application owns quality (the VMAF term) and is credited for the latency, jitter, and loss experienced over the window its bitrate governed; the transport agent is credited only for how well it delivered each frame. This assigns each decision the consequences it can actually control – the encoder is not blamed for a poor split, nor the scheduler for an over-ambitious bitrate – while the shared bottleneck still couples them: a bad split shows up as latency/loss in the application’s reward, and an over-high bitrate shows up as loss the transport agent cannot schedule away.

7.5 Continuous Scoring over a Variable Path Set

The transport agent inherits Part I’s problem – a variable, possibly changing number of paths – and its solution, adapted from a discrete choice to a continuous allocation. A shared per-path encoder maps each (global, path_{*i*}) pair to a logit; the logits are softmaxed into the split, so the policy is permutation-equivariant and independent of the path count. To handle paths that appear and disappear, each path carries a *liveness* flag and the state includes a binary mask; the softmax excludes dead paths, so traffic is never allocated to a path that is gone. The critic aggregates per-path embeddings with a masked mean-pool – a Deep Sets [23] operation – yielding a value estimate invariant to path ordering and count. This continuous scoring agent is the natural sibling of Part I’s discrete scoring selector: the same architectural idea, once for “pick a path” and once for “divide across paths.”

8 Implementation: Multipath Real-Time Video Testbed

Realizing the Part II design requires a testbed that is faithful enough to expose real multipath and congestion behavior yet fast enough for reinforcement-learning rollouts. We resolve this tension with a single data-plane abstraction served by two interchangeable backends (§8.1): a packet-level NS-3 simulator and a fast analytical mock. Section 8.2 describes the real-time video and multipath model, §8.3 the agent implementation and the VMAF surrogate, and §8.4 the known gaps that scope the current results.

8.1 Data-Plane Abstraction and Two Backends

Both backends implement one abstract `DataPlane` interface, so the RL code, observations, and rewards are written once and run unchanged against either. This lets us iterate quickly on the mock and validate on the higher-fidelity simulator without divergence.

NS-3 backend. A packet-level simulation in which the connection’s several paths are modeled as independent subflows over a topology of links, each with its own capacity, propagation delay, and active-queue management (FqCoDel). Realistic contention comes from OnOff UDP cross-traffic with exponentially distributed on/off periods, phase-shifted per path so the paths do not degrade in lockstep. Each video frame’s bytes are striped across the subflows according to the transport agent’s split, and delivery is tracked by a byte-watermark scheme – avoiding per-packet tagging – from which per-frame latency, jitter, and deadline-miss loss are measured. Python and NS-3 communicate through an `ns3-ai` shared-memory bridge.

Mock backend. A pure-Python, per-seed-deterministic model in which each path has a capacity envelope (a smooth sinusoidal component plus bursty cross-traffic and noise) and a standing-queue model that produces bufferbloat when a path is overdriven. It runs orders of magnitude faster than NS-3 and is used for rapid training and for scenarios the NS-3 body does not yet emit.

Non-stationary dynamics. The mock backend can optionally inject the volatility that makes multipath scheduling matter: *path churn* (paths appear and disappear,

and bytes sent to a dead path are lost), *regime shifts* (abrupt changes in the best path’s capacity), *congestion bursts* (transient per-path capacity collapses), and *correlated failures* (shared-bottleneck groups that degrade together). These are off by default and are the conditions under which Chapter 9 finds the learned split decisive.

8.2 Real-Time Video and Multipath Model

The video source mirrors a conferencing encoder: a fixed frame rate (30 fps by default), variable frame sizes (with periodic larger I-frames and per-frame jitter), no playback buffer, and a deadline after which a late frame is discarded as lost. The aggregate target bitrate ranges over 300 – 6000 kbit/s. The canonical topology is deliberately *non-dominant*: no single path can carry full-quality video, and the aggregate usable capacity sits below the quality knee, so the system must genuinely combine paths and trade rate against delivery. For example, a four-path configuration mixes paths of differing capacity and RTT – including a higher-capacity but high-latency “trap” path – under heavy, path-specific cross-traffic, so that neither the widest nor the lowest-latency path is right on its own.

8.3 Agents and QoE Surrogate

Agents. Both agents are soft actor-critic learners with twin critics and automatic temperature tuning. The transport agent has two variants: a *flat* version whose actor and critic consume a fixed-length concatenation of the global and per-path features (a fixed path count), and a *scoring* version implementing the permutation-equivariant, mask-aware design of §7.5 with a shared per-path actor and a Deep Sets masked-mean-pool critic. The hierarchical training loop runs the two agents at their respective cadences, storing each agent’s transitions in its own replay buffer and crediting the application agent over the window of frames its bitrate governed.

VMAF surrogate. The quality term of Eq. (7.1) needs a mapping from an encoding (and network conditions) to perceptual quality. Two are supported: a default bitrate-only logarithmic curve fit through published quality anchors, and a learned surrogate that interpolates a quality-of-service→VMAF table over bitrate, loss, delay, and jitter (produced by a sibling data-generation project). When the learned surrogate is active, the explicit loss penalty in Eq. (7.1) is dropped to avoid double-counting the loss it already reflects.

8.4 Known Gaps

The current testbed has three limitations that scope the results of Chapter 9 and define concrete follow-up work.

- **Dynamics parity.** The non-stationary dynamics (churn, regime shifts, bursts) are currently emitted only by the mock backend; porting them into the NS-3 body – adding per-path liveness to the simulation – is required before the packet-level and analytical results can be compared on equal footing. *[TODO: report NS-3 vs. mock parity once the C++ dynamics land.]*
- **Transport realism.** Subflows are modeled with TCP-like congestion control rather than true Multipath QUIC; this preserves congestion, queueing, and loss dynamics but masks some network loss as latency. Swapping in a real QUIC module is future work.
- **Surrogate coverage.** The learned VMAF surrogate’s loss, delay, and jitter axes are currently under-sampled, so the quality term is effectively bitrate-dominated; a richer surrogate would activate those axes and let quality respond to delivery conditions directly.

9 Evaluation: Joint Multipath Optimization

This chapter evaluates the Part II system. Section 9.1 gives the setup and baselines; §9.2 reports the central finding – that the value of learned multipath scheduling depends sharply on how volatile the network is; and §9.3 isolates the contributions of the joint design and the scoring transport agent. The numbers reported here are from preliminary runs on the mock backend; packet-level NS-3 confirmation is pending (§8.4) and is marked where relevant.

9.1 Setup and Baselines

Scenarios. We use two contrasting scenarios. The *static* scenario is the non-dominant four-path topology of §8.2, with stationary (if heavily loaded) paths. The *dynamic* scenario is a six-path setting with the non-stationary dynamics of §8.1 enabled – path churn, regime shifts, and congestion bursts – so that the identity and quality of the best paths change during a call.

Metric. The primary metric is mean per-window QoE (Eq. (7.1)); we also report the underlying achieved bitrate, latency, loss, and VMAF.

Baselines. The learned pair (application agent + transport agent) is compared against fixed scheduling policies driving the *same* learned bitrate controller where applicable: *even* split across live paths, *single* best-active path, and *proportional* allocation by capacity. The decisive ablation is *app-only*: the learned bitrate controller with a naive even split, isolating the value of the learned transport agent.

9.2 Results: When Does Splitting Help?

The headline result is that the learned split’s value is conditional on volatility. Table 9.1 summarizes preliminary QoE across scenarios and methods.

Two observations stand out. First, in the **static** scenario the learned pair (0.633) beats the single-best-path baseline (0.514) by roughly 23%, but the app-only ablation (0.665) is actually on par with – even marginally above – the full system.

Table 9.1: Preliminary mean QoE (Eq. (7.1)) by scenario and method (mock back-end). Higher is better. In the static scenario the split barely matters – the learned transport agent adds little over app-only – whereas in the dynamic scenario the split is decisive and app-only collapses.

Method	Static 4-path	Dynamic 6-path
Learned pair (app + transport)	0.633	0.606
App-only (learned rate, even split)	0.665	−0.212
Single best-active path	0.514	0.402
Proportional (by capacity)	[TODO: fill]	0.209
Even split	[TODO: fill]	0.109

When paths are stable, a good bitrate controller over a simple split captures nearly all the achievable QoE, and the transport agent has little left to earn. A practitioner reading only the static numbers might reasonably conclude that learned scheduling is not worth its complexity.

Second, the **dynamic** scenario overturns that conclusion. The learned pair (0.606) again leads, but now the app-only ablation *collapses* to −0.212: with paths churning and bursting, a fixed even split routes traffic onto dead or congested paths, driving latency and loss up despite a well-chosen bitrate. The fixed-schedule baselines degrade in order of how much they react to path state – single-best (0.402), proportional (0.209), even (0.109) – and only the learned transport agent, which observes path liveness and reallocates, sustains high QoE. This is the condition under which learned multipath scheduling is not a marginal optimization but the difference between a usable and an unusable call.

[TODO: add variance/confidence intervals over seeds, and a time series from a representative dynamic episode showing the transport agent reallocating away from a path as it churns or bursts (the *dyn-scoring* run illustrates convergence from strongly negative early-episode reward to stable positive QoE with latency in the tens of milliseconds and loss a few percent).]

Placeholder — Figure: QoE over training episodes for the learned pair vs. baselines in the dynamic scenario, and a within-episode trace of the per-path split tracking path liveness.

9.3 Ablation Study

Joint vs. app-only. Table 9.1 already provides the core ablation: removing the learned transport agent (app-only) is nearly free when the network is stable but

catastrophic when it is volatile. This quantifies the design premise of §7.1 – the two decisions must be co-optimized precisely in the regimes where the network is hard.

Flat vs. scoring transport. We compare the flat transport agent (fixed path count) against the permutation-equivariant scoring agent (§7.5) on the dynamic scenario, where the number of live paths changes. *[TODO: report the flat-vs-scoring comparison; the expected finding is that only the scoring/mask-aware agent handles a changing path count gracefully, since the flat agent cannot represent appearing/disappearing paths.]*

Backend parity. All numbers above are from the mock backend. Confirming them on the packet-level NS-3 backend requires the dynamics port of §8.4. *[TODO: report NS-3 results for the static scenario now, and for the dynamic scenario once the C++ dynamics land, to establish that the mock findings transfer to packet-level simulation.]*

9.4 Summary

Preliminary results support the joint, multipath design: co-optimizing bitrate and per-path split matches or beats every fixed-scheduling baseline, and the advantage of learned scheduling grows from negligible in a stable network to decisive under churn and bursts. The main open items are seed-level statistics, the flat-vs-scoring comparison, and NS-3 confirmation.

Part III

Synthesis and Outlook

10 Analysis

This chapter analyzes the two systems along the dimensions that determine whether learned path control is practical, with respect to the concerns that actually govern deployment: overhead (§10.1), scalability (§10.2), generalization (§10.3), and deployability (§10.4). Where a claim depends on measurements still in progress, it is marked.

10.1 Overhead

Two kinds of overhead matter: measurement and computation.

Measurement (probing) overhead. In Part I, probing is a first-class cost charged in the reward (Eq. (4.2)) and accounted per decision (§5.3). The learned selector probes only the paths it inspects, in contrast to heuristics that require a metric on every path, so its measurement footprint is intended to be a fraction of an exhaustive scan at comparable selection quality – the property we test in §6.4. In Part II, per-path state is derived from ordinary transport feedback (RTT samples, acknowledgements, loss signals) rather than dedicated probes, so the marginal measurement cost of the split decision is low.

Computational overhead. Both policies are small neural networks evaluated once per decision. The Part I selector runs at path-selection granularity (seconds to minutes) and is comfortably cheap. The Part II transport agent is the tighter constraint: it runs *per frame* (≈ 33 ms at 30 fps), so its inference must fit well inside a frame interval. The scoring architecture keeps this affordable – a shared per-path encoder evaluated N times and a softmax – but the per-frame budget is the binding real-time constraint. *[TODO: report measured per-decision inference latency for both agents, and confirm the transport agent’s inference fits within the frame budget on the target hardware.]*

10.2 Scalability

The scoring architecture is what makes both systems scale in the number of paths. Because a shared network scores each path independently and decisions are made by comparison (§4.4) or softmax (§7.5), the parameter count is independent

of N , and inference is linear in N . This lets one trained model serve source–destination pairs with any number of paths and topologies of any size, without retraining or padding to a fixed maximum – the limitation of the flat agents. The remaining scalability question is training coverage: the environment must expose enough variety of path counts, topologies, and conditions for the shared scorer to generalize. *[TODO: report how selection quality holds as topology size and path count grow, ideally on topologies larger than those seen in training.]*

10.3 Generalization

Generalization is required along three axes. **Across intents:** Part I’s FiLM conditioning is meant to interpolate to intents between (and, ideally, beyond) the trained reward profiles, so that a novel weight vector yields sensible behavior without retraining – tested in §6.3. **Across topologies and path counts:** the permutation-equivariant design gives structural generalization, but only empirical results can confirm it holds off the training distribution. **Across conditions:** the dynamic scenario of Part II (§9.2) is itself a generalization stress test, since the policy must cope with churn and bursts rather than a stationary environment; the collapse of the fixed-schedule baselines there shows that static policies do *not* generalize across volatility, which is the gap learned control fills. *[TODO: quantify off-distribution degradation for both systems – unseen intents, larger topologies, and unseen dynamics regimes – to bound how gracefully performance decays.]*

10.4 Deployability

10.4.1 Deployment Gaps

Several gaps still separate these prototypes from a deployment.

- **From simulation to reality.** Both systems are trained in simulation. Part I relies on a synthetic traffic and beaconing environment; Part II on NS-3 and a mock transport. Bridging the gap to deployment would require sim-to-real transfer – for example through domain randomization or online fine-tuning against live measurements – and the packet-level NS-3 backend is the intended stepping stone (§8.4).
- **Real transport.** Part II models subflows with TCP-like congestion control rather than true Multipath QUIC [5]. A real QUIC integration is therefore needed to validate the per-frame scheduling under an actual multipath transport.
- **Online and safe learning.** A deployed agent would learn or adapt against live traffic, where exploration has real cost. Bounding worst-case behavior –

for example by falling back to a safe heuristic when the policy is uncertain, or by constraining exploration – is a prerequisite that this work does not address.

Two properties help on the positive side. First, the endpoint-driven nature of SCION path selection means these controllers require no network-internal changes. Second, the horizon-agnostic application agent (§7.2) is designed for unbounded real sessions rather than fixed-length episodes.

10.4.2 Deployment and Trust Assumptions

This thesis is a performance and control study, not a security study, so in place of a classical attacker model we state the trust assumptions under which the learned controllers operate and note where security considerations would enter a real deployment.

- **Honest measurements.** We assume that the per-path measurements an agent acts on – whether obtained by active probing (Part I) or transport feedback (Part II) – are honest. A malicious on-path element that fabricates favorable metrics could steer the agent toward a bad path; defending against that is out of scope, although the trust term below offers partial mitigation.
- **SCION path authenticity and trust.** SCION’s cryptographically authenticated paths mean an agent knows *which* ASes a path traverses and cannot be silently redirected. Part I’s reward includes a *trust* component derived from path quality, allowing the agent to prefer higher-confidence paths and discount those with anomalous loss or latency – a lightweight hedge against unreliable or adversarial path advertisement, not a full defense.
- **No adversarial control of the learner.** We assume the agent’s own training and inference pipeline is trusted; adversarial manipulation of the RL policy (e.g. reward poisoning or observation attacks) is beyond this work.

A production deployment would need to revisit these assumptions – in particular by verifying measurement integrity and by bounding the damage a single misreported path can do.

11 Discussion

This chapter steps back from the individual results to their broader meaning: what the two parts jointly say about intent-driven network control (§11.1), why the findings matter beyond the specific systems (§11.2), and what the results cannot tell us (§11.3).

11.1 Interpretation

Read together, the two parts tell a consistent story about *when* additional control machinery earns its keep.

Intent conditioning turns a family of policies into one. Part I’s central lesson is that the right way to serve many objectives is not many models but one model that takes the objective as input – provided the objective is allowed to *modulate* the decision rather than merely accompany it. The concatenation-versus-FiLM contrast (§4.5) is the crisp version of this point: a conditioning signal that enters symmetrically across the alternatives cancels under the comparison that picks the action, so the policy can look conditioned while behaving identically. Feature-wise modulation avoids this because it changes the *relative* salience of path features, and thus which path wins. This is a design principle, not a SCION-specific trick: any comparison-based selector conditioned on an objective should modulate its per-alternative features.

The value of multipath control is a function of volatility. Part II’s central lesson is that learned multipath scheduling ranges from nearly worthless to decisive depending on the network. In the static scenario, a good bitrate controller over a trivial split captured essentially all achievable QoE, and the learned transport agent added little (§9.2). In the dynamic scenario, the same app-only configuration collapsed to negative QoE while the learned split sustained a usable call. The practical implication is that one should not evaluate multipath schedulers only in stable conditions – that is precisely the regime in which they look unnecessary – and that the case for learned scheduling rests on volatility, churn, and bursts, where static policies cannot adapt.

One principle, two layers. Both parts instantiate the Mortise principle [17] – intent as a first-class input that reshapes a network mechanism – at successively

deeper points of control. Part I reshapes *which path* is chosen; Part II reshapes both *how the paths are used* and *what the application produces*. The progression suggests a general recipe: expose the application’s intent, and let it steer every controllable decision between the application and the wire.

11.2 Implications

The results matter for how we build controllers on path-aware architectures. Prior learned path selectors [3] and learned congestion or scheduling controllers [9, 24, 21] each optimize a fixed objective; our work argues that on a path-aware substrate the objective should be a knob, and that a single conditioned policy can replace a zoo of per-objective models. For real-time media specifically, the joint bitrate-and-split result reinforces a theme from learned ABR [11] – that application and network decisions are better made together – and extends it to the multipath setting that SCION makes routine. More broadly, the two systems are evidence that SCION’s endpoint path choice is not merely a routing convenience but an enabler of application-aware optimization that is awkward or impossible in a destination-based Internet.

11.3 Limitations

The results are bounded in ways that the analysis (Chapter 10) details and that fairness requires restating here. Both systems are evaluated in simulation, and Part II’s headline dynamic-scenario numbers are from the mock backend pending NS-3 confirmation (§8.4). Intent in this thesis is *declared*, not *inferred*: we assume the application can state its objective, and we do not study how to recover a latent intent from traffic – a natural but separate problem. The transport model is TCP-like rather than true Multipath QUIC, and the VMAF surrogate is bitrate-dominated, so absolute QoE figures should be read as relative comparisons rather than calibrated user scores. Finally, we have not addressed online, safe learning against live traffic, which any deployment would require. None of these undercut the two qualitative findings – modulation-based conditioning works where concatenation fails, and multipath control pays off in proportion to volatility – but they do delimit how far the quantitative results should be pushed.

12 Related Work

Where Chapter 2 covered the systems this thesis *builds on*, this chapter surveys work that attacks the *same or similar problems* and positions our contributions against it. We group it into learned routing and traffic engineering (§12.1), learned congestion control and multipath scheduling (§12.2), adaptive bitrate and QoE-driven control (§12.3), goal-conditioned and modulated policies (§12.4), and intent-based networking (§12.5).

12.1 Learned Routing and Traffic Engineering

A line of work applies reinforcement learning to routing and traffic engineering. *Learning to Route* [18] studies RL and learned representations for routing, and experience-driven networking [22] uses deep RL for traffic-engineering decisions. These target network-operator objectives (e.g. minimizing congestion globally) from a control-plane vantage point. Our Part I differs in both the actor and the objective: the decision is made by an *endpoint* choosing among cryptographically authenticated SCION paths, and the objective is the *application's* intent rather than a global network utility. Most directly related is the DQN SCION path selector of Bourak et al. [3], which we extend; the differences – a variable-action scoring architecture and FiLM-based intent conditioning that removes the single-objective limitation – are the substance of Part I.

12.2 Learned Congestion Control and Multipath Scheduling

Learned transport controllers include Aurora [9], which casts congestion control as RL, and Orca [1], which combines a learned layer with a classic controller. Mortise [17] is the closest in spirit: it auto-tunes congestion-control parameters to the running application's QoE, and its principle – intent as an input that reshapes a mechanism – is the template we transplant into path selection and joint multipath control. For multipath *scheduling*, ReLeS [24] learns a Multipath QUIC scheduler and Peekaboo [21] learns to schedule across heterogeneous paths in dynamic environments; both optimize a scheduling policy for a given, exogenous load. Part II's distinction is that it does *not* treat the offered load as fixed: it co-optimizes the application's bitrate with the split, so the controller can relieve congestion it

would otherwise create – a coupling those schedulers, and the Multipath TCP mechanisms of RFC 8684 [6], do not address.

12.3 Adaptive Bitrate and QoE-Driven Control

Learned adaptive bitrate control, exemplified by Pensieve [11], showed RL beating hand-crafted ABR heuristics for buffered video streaming, using QoE-shaped rewards and quality metrics such as VMAF [10]. Our Part II shares the QoE-driven, learned-bitrate stance but targets *real-time* video (no playback buffer, deadline-based loss) and, crucially, couples the bitrate decision to a simultaneous multipath scheduling decision. Where Pensieve adapts rate to a single, exogenous network path, we adapt rate *and* decide how to spread it across several paths, jointly.

12.3.1 Mortise: intent-driven tuning of a transport mechanism

The conceptual template for both parts is Mortise [17]. Mortise observes that a single congestion-control algorithm cannot be optimal for every application, and instead *auto-tunes* the algorithm’s parameters to maximize the running application’s QoE, using the application’s revealed preferences over throughput, latency, and loss to steer the tuning. The essential idea – treat application intent as a first-class input that reshapes a network mechanism rather than a constant compiled into it – is exactly what we transplant. Part I applies it to *path selection* (which path to use), and Part II applies it to *joint rate and multipath scheduling* (what the application sends and how it is spread across paths).

12.4 Goal-Conditioned and Modulated Policies

The machinery for serving many objectives with one policy comes from goal-conditioned RL – universal value function approximators [15] and hindsight relabeling [2] – and from conditioning mechanisms such as FiLM [13], originally introduced for visual reasoning. Our contribution is not the mechanism but its application and a specific insight about *where* it is necessary: for a comparison-based (argmax or softmax) selector, we show conditioning must modulate the per-alternative features, because an additive, symmetric signal cancels under the comparison (§4.5). This connects a general representation-learning tool to a concrete failure mode in networked decision-making.

12.5 Intent-Based Networking

Finally, “intent” also names a body of network-management work concerned with translating high-level operator goals into configuration, surveyed in RFC 9315 [4].

That community’s “intent” is an operator’s declarative policy for a whole network; ours is an *application’s* per-flow QoS preference consumed by a learned endpoint controller. The two are complementary rather than competing – one could imagine operator intent bounding the space in which application intent is served – but they operate at different layers and timescales, and our techniques address the fine-grained, per-flow end of the spectrum.

13 Conclusion

This thesis asked how an endpoint on a path-aware Internet should choose the best use of its paths given the intention of the application and the state of the network. It answered in two movements that share one idea, drawn from Mortise [17]: treat application intent as a first-class input that reshapes a network mechanism, not as a constant compiled into it.

Part I turned a fixed-objective DQN path selector for SCION [3] into an intent-conditioned one. Two design decisions carried the work: a permutation-equivariant *scoring* architecture that handles a variable, changing set of candidate paths, and a *feature-wise modulation* of path features by the intent vector that lets a single policy serve the whole spectrum of objectives without retraining. Part II asked what happens if we use several paths at once and optimize the application itself, and built a two-timescale hierarchical agent pair that co-optimizes video bitrate and per-path traffic split over Multipath QUIC, coupled so that each agent accounts for the other.

What we learned. The most useful findings were the ones that cut against intuition. First, that *how* intent is injected matters more than *that* it is injected: for a comparison-based selector, appending the intent to the state leaves decisions unchanged, because a symmetric signal cancels under the argmax; only modulation reorders the alternatives. Second, that the value of learned multipath scheduling is *not* a fixed quantity but a function of network volatility – negligible when paths are stable, where a good bitrate controller over a trivial split suffices, yet decisive under churn and bursts, where the same app-only configuration collapses. Both findings sharpen when, and when not, to reach for the more sophisticated mechanism.

Future work. Several threads remain open, and much of the draft’s structure is deliberately laid out to guide them. On the experimental side: completing the Part I baseline, intent-alignment, and ablation tables; adding seed-level statistics and the flat-versus-scoring comparison in Part II; and porting the non-stationary dynamics into the NS-3 backend so packet-level results can confirm the mock findings. On the system side: replacing the TCP-like subflows with a real Multipath QUIC transport, and enriching the VMAF surrogate so quality responds to delivery conditions, not just bitrate. On the research side, three directions stand out: *inferring* intent from traffic rather than assuming it is declared; *online and safe* learning against live traffic with bounded worst-case behavior; and pushing the “one principle, every

controllable decision” recipe further – for instance, conditioning the Part II agents on an explicit intent vector as Part I does, unifying the two systems into a single intent-conditioned, cross-layer controller for path-aware networks.

Bibliography

- [1] S. Abbasloo, C.-Y. Yen, and H. J. Chao. Classic meets modern: A pragmatic learning-based congestion control for the Internet. In *Proceedings of the ACM SIGCOMM Conference*, pages 632–647, 2020.
- [2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [3] O. Bourak, T. Burman, T. John, H. Daltrophe, T. Shrot, and D. Hausheer. Path optimization using DQN in the SCION Internet architecture. In *Cyber Security, Cryptology, and Machine Learning (CSCML 2025)*, volume 16244 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.
- [4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura. Intent-based networking – concepts and definitions. RFC 9315, Internet Engineering Task Force (IETF), 2022.
- [5] Q. D. Coninck and O. Bonaventure. Multipath QUIC: Design and evaluation. In *Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, pages 160–166, 2017.
- [6] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch. TCP extensions for multipath operation with multiple addresses. RFC 8684, Internet Engineering Task Force (IETF), 2020.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [8] J. Iyengar and M. Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, Internet Engineering Task Force (IETF), 2021.
- [9] N. Jay, N. H. Rotman, B. Godfrey, M. Schapira, and A. Tamar. A deep reinforcement learning perspective on Internet congestion control. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3050–3059, 2019.

- [10] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara. Toward a practical perceptual video quality metric. Netflix Technology Blog, 2016. <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>.
- [11] H. Mao, R. Netravali, and M. Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the ACM SIGCOMM Conference*, pages 197–210, 2017.
- [12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [13] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, pages 3942–3951, 2018.
- [14] A. Perrig, P. Szalachowski, R. M. Reischuk, and L. Chuat. *SCION: A Secure Internet Architecture*. Springer, 2017.
- [15] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1312–1320, 2015.
- [16] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [17] Y. Shen, R. Chen, B. Wang, J. Chen, H. Zhang, M. Wang, Y. Liu, M. Xu, and Z. Meng. Mortise: Auto-tuning congestion control to optimize QoE via network-aware parameter optimization. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026.
- [18] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar. Learning to route. In *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 185–191, 2017.
- [19] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, pages 2094–2100, 2016.
- [20] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1995–2003, 2016.

- [21] H. Wu, O. Alay, A. Brunstrom, S. Ferlin, and G. Caso. Peekaboo: Learning-based multipath scheduling for dynamic heterogeneous environments. *IEEE Journal on Selected Areas in Communications*, 38(10):2295–2310, 2020.
- [22] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang. Experience-driven networking: A deep reinforcement learning based approach. In *IEEE INFOCOM 2018 – IEEE Conference on Computer Communications*, pages 1871–1879, 2018.
- [23] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [24] H. Zhang, W. Li, S. Gao, X. Wang, and B. Ye. ReLeS: A neural adaptive multipath scheduler based on deep reinforcement learning. In *IEEE INFOCOM 2019 – IEEE Conference on Computer Communications*, pages 1648–1656, 2019.

A Reproducibility Details

This appendix collects the hyperparameters, reward-profile definitions, and supplementary results that support the main text. Values still to be finalized from the experiment runs are marked.

A.1 Part I: Reward Profiles

The conditional path-selection agent is trained and evaluated across the reward-weight vectors $\mathbf{w} = (w_1, w_2, w_3, w_4, w_{\text{probe}})$ of Eq. (4.2), spanning distinct application intents. Table A.1 lists representative profiles.

Table A.1: Representative reward profiles (application intents) used in Part I. w_1 weights goodput, w_2 trust, w_3 loss and w_4 delay within the trust term, and w_{probe} the probing penalty.

Profile	w_1	w_2	w_3	w_4	w_{probe}
Balanced	0.70	0.30	0.50	0.50	0.05
Throughput	0.95	0.05	0.35	0.35	0.05
Trust / quality	0.15	0.85	0.85	0.85	0.05
Latency	0.45	0.55	0.25	0.95	0.05
Low probe cost	0.70	0.30	0.50	0.50	0.005
High probe cost	0.70	0.30	0.50	0.50	0.25

A.2 Part I: Agent Hyperparameters

[TODO: tabulate the final hyperparameters (hidden sizes, learning rate, discount γ , target-update rate, batch size, replay capacity, PER α/β schedule, number of training episodes) for the flat, scoring, and conditional agents.]

A.3 Part II: QoE Weights and SAC Hyperparameters

The QoE reward of Eq. (7.1) uses default weights $(a, b, c, d) = (1, 0.5, 0.5, 1)$ with latency and jitter normalizers of 200 ms and 50 ms; the application agent acts every 1 s and the transport agent every frame (≈ 33 ms at 30 fps) over a bitrate range of

300 – 6000 kbit/s. **[TODO:** *tabulate the SAC hyperparameters (network sizes, learning rates, discount, target entropy, temperature schedule, replay capacity, batch size) and the exact topology parameters (per-path capacity, RTT, cross-traffic intensity) for the static four-path and dynamic six-path scenarios.*]

A.4 Supplementary Results

[TODO: *additional plots: per-episode training curves, per-intent path-choice distributions (Part I), and representative within-episode split traces (Part II).*]